Detailed Research Methodology

Spatiotemporal Graph Neural Networks for Real-Time Fault Localization in Smart Grids

Using the IEEE 39-Bus (New England) Test System

Anik Kumar Basu (22234103041) | Sabrina Shonda Snaha (22234103048) | Md. Sadik Mahmud Adive (22234103057)

1. Methodology Overview

This document presents the complete, step-by-step research methodology for building and validating a Spatiotemporal Graph Neural Network (ST-GNN) to detect and localize electrical faults in the IEEE 39-bus (New England) test system. The methodology follows a structured pipeline from dataset generation through model deployment evaluation, ensuring reproducibility and scientific rigor.

The IEEE 39-bus system is selected as the primary benchmark because it represents a realistic, medium-scale transmission network with 39 buses, 46 transmission lines, 10 generators, and 19 load buses — complex enough to capture real-world topological dependencies yet tractable for academic research.

Methodology Pipeline at a Glance:

Phase	Activity	Key Output
Phase 1	Grid Modeling & Graph Construction	Graph G = (V, E), Adjacency Matrix A
Phase 2	Synthetic Fault Data Generation	Labeled dataset of 50,000+ samples
Phase 3	Data Preprocessing & Feature Engineering	Normalized node feature tensors
Phase 4	ST-GNN Architecture Design	PyTorch Geometric model
Phase 5	Model Training & Hyperparameter Tuning	Trained weights, learning curves
Phase 6	Evaluation & Benchmarking	Accuracy, F1, localization error
Phase 7	Robustness Analysis	Noise tolerance curves

2. Phase 1 — IEEE 39-Bus Graph Construction

2.1 Power Grid as a Graph

The IEEE 39-bus New England system is formally represented as an undirected weighted graph G = (V, E, X, W) where:

- V = {v_1, v_2, ..., v_{39}} is the set of 39 nodes (buses/substations)

- E ⊆ V × V is the set of 46 edges (transmission lines and transformers)
- X ∈ ℝ^(39 × F) is the node feature matrix (F features per node at each timestep)
- W ∈ ℝ^(39 × 39) is the weighted adjacency matrix encoding line impedances

2.2 Adjacency Matrix Construction

The adjacency matrix A is constructed from the standard IEEE 39-bus network topology. Two forms are used:

- Binary adjacency: A[i,j] = 1 if bus i and bus j are directly connected by a line, 0 otherwise.
- Weighted adjacency: A[i,j] = 1 / (R_ij + jX_ij), the inverse of line impedance, reflecting electrical proximity. This ensures that lines with lower impedance (stronger coupling) carry higher weight in the GNN message passing.

The normalized graph Laplacian used in GCN propagation:

$$\tilde{A} = \tilde{D}^{(-1/2)} \cdot (A + I) \cdot \tilde{D}^{(-1/2)}$$

where I is the identity matrix (self-loops) and D̃ is the degree matrix of (A + I). This normalization prevents numerical instability from summing node features across high-degree nodes.

2.3 Node Feature Set

Each node v_i at timestep t carries the following feature vector x_i(t) ∈ ℝ⁵:

Feature	Symbol	Unit	Physical Meaning
Voltage Magnitude	Vm	p.u.	Deviation from nominal voltage (1.0 p.u.)
Voltage Phase Angle	θ	degrees	Phase shift at the bus
Active Power Injection	P	MW	Net generation minus load
Reactive Power Injection	Q	MVAR	Reactive power balance
Frequency Deviation	Δf	Hz	Deviation from 60 Hz nominal

These five features capture the electrical state of each bus comprehensively. Voltage anomalies and power flow disruptions are primary indicators of fault propagation through the network.

3. Phase 2 — Synthetic Fault Dataset Generation

3.1 Simulation Environment

Since real-world fault event data from power utilities is confidential and scarce, a labeled synthetic dataset is generated using Pandapower (Python library v2.13+), which implements the Newton-Raphson power flow solver compliant with IEEE standards.

- Base case: IEEE 39-bus network loaded from pandapower.networks.case39()
- Fault simulation: Short-circuit analysis via pandapower.shortcircuit module
- Simulation timestep: 10 ms (100 Hz sampling rate)

- Pre-fault window: 200 ms (20 samples) of normal operation
- Post-fault window: 300 ms (30 samples) capturing transient response

3.2 Fault Type Taxonomy

Four standard fault types are simulated, covering the most common fault modes in power transmission:

Fault Type	Abbrev.	Description	% of Real Faults
Single Line-to-Ground	SLG	One phase contacts ground. Most common fault.	~70%
Line-to-Line	LL	Two phases contact each other.	~15%
Double Line-to-Ground	DLG	Two phases contact ground simultaneously.	~10%
Three-Phase Symmetric	3PH	All three phases short-circuit. Most severe.	~5%

3.3 Dataset Composition

Faults are simulated at each of the 39 buses, under varying fault resistance (Rf = 0, 5, 10, 20 Ω) and different load levels (light: 60%, nominal: 100%, heavy: 130%). This yields a rich, diverse dataset:

Condition	Count
Total fault simulations	39 buses × 4 fault types × 4 resistances × 3 load levels = 1,872 scenarios
Samples per scenario (time-series windows)	~27 per scenario
Total fault samples	~50,544 labeled fault samples
Normal operation samples (no fault)	10,000 samples
Total dataset size	~60,000 samples
Train / Validation / Test split	70% / 15% / 15%

Each sample is a tuple (X_t, y_loc, y_type) where X_t ∈ ℝ^(39 × 5 × T) is the spatiotemporal node feature tensor over a sliding window of T = 50 timesteps, y_loc ∈ {0,...,38} is the faulted bus index, and y_type ∈ {0,1,2,3} is the fault type class.

4. Phase 3 — Data Preprocessing & Feature Engineering

4.1 Normalization

All node features are normalized using Z-score standardization computed only on the training set to prevent data leakage:

$$\hat{x} = (x - \mu_{train}) / \sigma_{train}$$

This ensures that voltage magnitudes (scale ~1.0 p.u.) and active power values (scale ~100s of MW) are on comparable scales, preventing large-magnitude features from dominating GNN weight updates.

4.2 Sliding Window Construction

A sliding window of T = 50 consecutive timesteps (500 ms at 100 Hz) is applied to each simulation run to create independent training samples. The window captures both the pre-fault baseline and the transient fault signature:

- Window stride: 5 timesteps (50 ms) — allows 80% overlap for data augmentation
- Each window: $X_t \in \mathbb{R}^{39 \times 5 \times 50}$ — 39 nodes, 5 features, 50 timesteps
- Label assignment: If fault onset occurs within the window, the window is labeled as a fault sample

4.3 Class Imbalance Handling

The dataset is mildly imbalanced (normal vs. fault, and across fault types). Two strategies are applied:

- Weighted cross-entropy loss: Class weights inversely proportional to class frequency are assigned during training.
- Oversampling of rare fault types (3PH): SMOTE-style augmentation in feature space to balance the 4 fault type classes.

4.4 Noise Augmentation

To improve real-world robustness, Gaussian noise with $\sigma \in \{0.01, 0.02, 0.05\}$ p.u. is added to node features during training, simulating sensor measurement errors and communication noise in smart grid infrastructure.

5. Phase 4 — ST-GNN Model Architecture

5.1 Architecture Overview

The proposed Spatiotemporal Graph Convolutional Network (ST-GCN) processes the 3D tensor $X_t \in \mathbb{R}^{39 \times 5 \times 50}$ through alternating spatial (GCN) and temporal (LSTM) processing blocks, culminating in dual output heads for fault localization and fault type classification.

5.2 Spatial Block: Graph Convolutional Network (GCN)

The GCN layer aggregates information from each node's direct neighbors, respecting the physical topology of the power grid. For each timestep t, the GCN propagation rule is:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{(-1/2)} \tilde{A} \tilde{D}^{(-1/2)} H^{(l)} W^{(l)} \right)$$

where $H^{(l)} \in \mathbb{R}^{39 \times d_l}$ are node embeddings at layer l, $W^{(l)}$ is the learnable weight matrix, and σ is the ReLU activation. Two stacked GCN layers are used, with a 2-hop receptive field capturing information from nodes up to 2 transmission lines away.

GCN Layer	Input Dim	Output Dim	Receptive Field
GCN-1	5 features	64 hidden	1-hop neighbors
GCN-2	64 hidden	128 hidden	2-hop neighbors

5.3 Temporal Block: LSTM

After the GCN extracts spatial node embeddings at each timestep, the sequence of embeddings H_1, H_2, ..., H_50 ∈ ℝ^(39 × 128) is processed by an LSTM to capture temporal dynamics:

- The LSTM receives per-node embeddings as sequences, treating each node independently across time.
- LSTM hidden size: 256 units, 2 layers with dropout (p=0.3) between layers.
- The final hidden state h_T ∈ ℝ^(39 × 256) encodes the cumulative spatiotemporal fault signature for all nodes.

5.4 Readout & Output Heads

A global mean pooling operation aggregates node-level representations into a graph-level embedding g ∈ ℝ^256. Two parallel output heads then perform:

- Fault Localization Head: A 3-layer MLP (256 → 128 → 64 → 39) with Softmax activation. Outputs a probability distribution over the 39 buses, where the argmax is the predicted faulted bus index.
- Fault Type Classification Head: A 3-layer MLP (256 → 64 → 4) with Softmax activation. Outputs probabilities for {SLG, LL, DLG, 3PH}.

This multi-task learning setup forces the shared ST-GNN backbone to learn representations jointly useful for both localization and classification, improving generalization.

5.5 Complete Architecture Summary

Component	Type	Parameters	Output Shape
Input	Node Feature Tensor	-	(39, 5, 50)
GCN-1 (per timestep)	Graph Convolution + ReLU	5×64 = 320	(39, 64, 50)
GCN-2 (per timestep)	Graph Convolution + ReLU	64×128 = 8,192	(39, 128, 50)
LSTM	2-layer LSTM, hidden=256	~1.3M	(39, 256)
Global Mean Pool	Aggregation	0	(256,)
Localization MLP	FC layers + Softmax	~50K	(39,)
Type MLP	FC layers + Softmax	~18K	(4,)

6. Phase 5 — Model Training & Optimization

6.1 Loss Function

A combined multi-task loss is used during training:

$$L_{total} = \alpha \cdot L_{localization} + (1-\alpha) \cdot L_{type\_classification}$$

where both L_localization and L_type_classification are weighted cross-entropy losses, and α = 0.7 places higher emphasis on the primary localization task.

6.2 Training Configuration

Hyperparameter	Value	Rationale
Optimizer	Adam (β_1=0.9, β_2=0.999)	Adaptive learning rates for sparse gradients
Initial Learning Rate	1×10^{-3}	Standard for GNN training
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=10)	Prevent stagnation
Batch Size	32 graph snapshots	GPU memory balance
Max Epochs	200	With early stopping (patience=20)
Dropout Rate	0.3 (LSTM), 0.2 (MLP)	Regularization
Weight Decay (L2)	1×10^{-4}	Prevent overfitting
GPU	NVIDIA RTX 3060 or equivalent	CUDA 11.8+ required

6.3 Baseline Models for Comparison

The ST-GNN is benchmarked against four baseline methods to rigorously demonstrate its superiority:

- Baseline 1 — Threshold-Based Rule System: Traditional SCADA-style detection using predefined voltage deviation thresholds (|Vm - 1.0| > 0.1 p.u.).
- Baseline 2 — Random Forest: Trained on flattened node features, ignoring topology.
- Baseline 3 — CNN-LSTM (Topology-Agnostic): Same temporal architecture but treats grid data as a regular 1D signal without graph structure.
- Baseline 4 — Vanilla GCN (No Temporal): Graph convolution only, no LSTM — captures topology but not temporal dynamics.

7. Phase 6 — Evaluation Metrics & Benchmarking

7.1 Primary Metrics

Metric	Formula	Target
Fault Detection Accuracy	TP+TN / (TP+TN+FP+FN)	> 95%
Fault Localization Accuracy (Top-1)	Correct bus / Total samples	> 90%
Fault Localization Accuracy (Top-3)	Correct bus in top-3 / Total samples	> 98%
Weighted F1-Score (Type)	Harmonic mean of precision & recall	> 0.92

Metric	Formula	Target
Mean Absolute Bus Error	Average |predicted_bus - actual_bus|	< 2 buses
Inference Latency	Time per sample on CPU	< 10 ms

Top-3 localization accuracy is particularly important for maintenance crew dispatch — knowing the fault is in one of three candidate buses is operationally sufficient to begin isolation.

7.2 Confusion Matrix Analysis

A 39×39 bus confusion matrix is generated to identify systematic localization errors. Buses that are frequently confused are expected to be electrically adjacent (connected by a single transmission line), validating that model errors are physically plausible rather than random.

7.3 Ablation Study

To quantify the contribution of each component, the following ablation experiments are conducted:

Experiment	Component Removed	Expected Δ Accuracy
Ablation 1	Remove LSTM (spatial only)	↓ ~8-12% localization acc.
Ablation 2	Remove GCN (temporal only, flat features)	↓ ~15-20% localization acc.
Ablation 3	Remove impedance weighting (binary A)	↓ ~3-5% localization acc.
Ablation 4	Remove fault type head (single-task)	↓ ~2-4% localization acc.
Ablation 5	Remove noise augmentation	↓ robustness on noisy test set

8. Phase 7 — Robustness & Generalization Analysis

8.1 Noise Sensitivity Testing

The trained model is evaluated on test sets with varying levels of added Gaussian noise (σ = 0.0, 0.01, 0.02, 0.05, 0.10 p.u.) to produce a noise-accuracy degradation curve. This simulates real-world sensor noise, communication errors, and measurement uncertainty in PMU data.

8.2 Fault Resistance Sensitivity

High-impedance faults (Rf > 20 Ω) are notoriously difficult to detect because they produce subtle voltage deviations. The model is specifically tested on this challenging regime to quantify detection limits.

8.3 Topological Perturbation (N-1 Contingency)

Real grids undergo planned and unplanned line outages. The model is tested on N-1 contingency scenarios (one line removed from the network) to assess whether the GCN's learned topology can

generalize to slightly different graph structures without retraining. This is a critical real-world requirement.

8.4 Cross-System Transfer (IEEE 14-Bus)

As an exploratory experiment, the model trained on the 39-bus system is fine-tuned on a small 14-bus labeled dataset to assess transfer learning potential across different grid topologies. This demonstrates whether the ST-GNN learns transferable physical principles rather than topology-specific patterns.

9. Implementation Details

9.1 Software Stack

Tool / Library	Version	Purpose
Python	3.9+	Primary programming language
PyTorch	2.1.0	Deep learning framework
PyTorch Geometric (PyG)	2.4.0	GNN layer implementations (GCNConv)
Pandapower	2.13.1	IEEE 39-bus simulation & fault generation
NumPy / SciPy	1.24 / 1.11	Numerical computation
Scikit-learn	1.3	Baseline ML models, metrics
Matplotlib / Seaborn	3.7 / 0.12	Visualization of results
Weights & Biases	0.16	Experiment tracking & hyperparameter logging

9.2 Reproducibility Measures

- All random seeds fixed: torch.manual_seed(42), numpy.random.seed(42)
- Full dataset generation scripts and model weights published to GitHub repository
- All experiments logged with Weights & Biases for hyperparameter traceability
- Docker container provided with pinned dependency versions

10. Project Timeline

Week	Activity	Deliverable
1-2	Grid modeling + Pandapower setup	IEEE 39-bus graph object + adjacency matrix
3-4	Fault simulation + dataset generation	50K+ labeled CSV samples
5	Preprocessing + data loader	PyG Data objects, train/val/test splits
6-7	ST-GNN implementation (GCN + LSTM)	PyTorch model code
8-9	Training + hyperparameter tuning	Trained model checkpoint

Week	Activity	Deliverable
10	Baseline experiments	Comparison table
11	Ablation studies	Component contribution analysis
12	Robustness + noise testing	Noise-accuracy curves
13	Visualization + confusion matrix	Figures and heatmaps
14	Report writing + code cleanup	Final research report

Document prepared for the course project submission | March 2026